NTNU | Faculty of Information Technology and Electrical Engineering
Department of Electronic Systems

Specialization Projects

Specialization Project fall 2026

Hyperspectral Data Compression Using Deep Learning

Author:
Peder Ingmar Dahl

Supervisor:
Prof. Milica Orlandic

table of contents

1 Introduction **2**

2 Theory **3**

2.1 Uniform Quantization 3

2.2 Quantization Granularity 3

2.3 Moment of Quantization 3

2.4 Straight Through Estimator 3

2.5 Non-uniform Quantization 4

2.6 Power of Two Quantization 5

2.7 (. 5

3 Method **6**

4 Results **7**

5 Discussion **8**

6 Conclusion **9**

References **10**

Appendixes **11**

Abstract

1 Introduction

The goal of this project is to investigate different quantization techniques for compressiong Convolutional Neural Networks (CNNs). Specifically CNNs that process hyperspectural data at edge devices are considered.

Quantization is an important technique often used when memory or computational resources are limited. For instance, quantization has beeing important in the recents advancements in LLMs, where the limitations in memory and compute has come from the rapidly increasing number of model parameters and the ever increasing demand for fast and reliable ai agents.

Quantization often come at the cost of computational accuracy. In general precision increaes linearly with the quantization noise, which increases a models accuracy in a predictible way. However when quantizing neural networks the the accuracy loss is not alloways as predicable. Models are often overparametericed which allows them to withstand precision loss without lossing as much accuracy. In addition to this, techniques can be used to allow a model to adapt to the quantization noise. Compressing these models becomes a optimization problem looking for the least overparametericed solution.

This project focuses on using quantization for compression models that will be deployed on resource constraind edge devices. Specificly on models that process hyperspectural data onboard satellites. The hypso mission... why we want to process onboard.. the downlink bottleneck in remote sensing - hyperspectural data is large and expensive to transmit.

We will specificly investigate mixed precision quantization... Non-uniform quantization using logarithmic quantization with power-of-two (PoT) and addetiv power-of-two (APoT) for model weights... ... techniques to measure quantization sensitivity of the model to quantization such as HAWQ.

This report will compare the performance by accuracy, size and computational efficiency.

2 Theory

2.1 Uniform Quantization

$$q = \text{clip}\big(\text{round}\big(\tfrac{x}{S}\big) + Z,\ q_{\min},\ q_{\max}\big)\,, \qquad q_{\min} = -2^{b-1},\ q_{\max} = 2^{b-1} - 1 \tag{1}$$

$$x_q = Q(x) = S(q - Z) \tag{2}$$

Z: zero point

2.2 Quantization Granularity

Per tensor quantization vs per channel quantization ...

2.3 Moment of Quantization

There are generaly two ways to quantize a neural network model based on when quantization occurs. The simplest way to do quantization is to train the model in full floating point precision and then quantize the weights and activations after training. This is called Post Training Quantization (PTQ). This is simple and efficient, but often hurts the models accuracy due to the quantization noise. When quantizing model activations using PTQ, this involves a calibrations step where calibration data is sendt through the network to to compute the clipping ranges and the scaling factors for the activation quantizer.

To regain some of the accuracy it is possible to apply quantization during training, also called Quantization Aware Training (QAT). This way the network can addapt to the quantization noise during training to find a better solution. QAT can be applyed from a checkpoint of a model already trained in full precision (called warmup), or from the first training iteration. A few iterations of warmup can sometimes help a model converge to a solution faster druing training.

Quantization Aware Training involves applying fake quantization. This means that the quantized parameters of the model are stored in FP32 and all hardware operations are performed in FP32, but before operations in the forward and backward pass rounding and clamping is performed to simulate the effects of quantization. The convertion to fixed point is performed after training like with PTQ.

The drawbacks of QAT is that it requires more computational resources. This is because the fake quantization step adds computation, there are more quantization-specific model parameters with associated gradient components that are involved in computations. The model often also requires more iterations to converge to a solution, the main reason for this is the Straight Through Estimator.

2.4 Straight Through Estimator

During QAT the weights are stored in full precision, and the quantized weights ($\hat{w}$) are calculated during the forward pass and used when deployed. The full precision weights (w) are often called the shadow weights. Backward computes $\frac{\partial L}{\partial w}$ and the optimizer applies it to the shadow weights; the quantized weights are only ever derived from them. The reason is that a gradient step must be able to move a weight by an arbitrarily small amount, and a quantized weight can only move one level at a time. Imagine updating a quantized weigth tensor: a too small step in the direction of a gradient will round back to the origional value, this freezes training. Updating on the shadow weights lets us keep the small gradiant steps done in the backward pass, which will eventully push the $\hat{w}$ across a rounding boundary.

The forward pass uses *hatw* which gives the well defined gradiant: $\frac{\partial L}{\partial \hat{w}}$, in the backward pass. The shadow weight gradient is given by (3).

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{w}} \cdot \frac{\partial \hat{w}}{\partial w} \tag{3}$$

The rounding applyed by the quantizer results in $\frac{\partial \hat{w}}{\partial w}$ being zero almost everywhere and undefined at the rounding and clipping thresholds. This means that the loss function is piecewise constant with respect to the quantized parameters. This again resulting in a gradient that either halt or explode training of those parameters.

The most common workaround for this problem is using a The Straight Through Estimator (STE). When calculating (3) in the backward pass, STE assumes $\frac{\partial \hat{w}}{\partial w} = 1$ which gives $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{w}}$. We say that the STE lets the gradiant from the output of the quantizer pass straight through to the input shadow weight during the backward pass. This gives us a well defined usefull gradiant, however it does lead to biased parapeter updates and limits how well the model can learn. The fewer bits used in quantization, the further appart are the rounding boundaries will be, making training harder.

2.5 Non-uniform Quantization

Uniform quantization means the quantization levels follow a uniform distribution over the quantization range. When this is not the case the quantization is called non uniform.

Non uniform quatnization can reduce the quantization loss if the variables being quantized follow a non-uniform distribution. Neural network model weights normally follow a gaussian distribution [], with higher density of weight values closer to zero. This mean that using uniform quantization wasts quantization levels for higher valued weights and has to few to capture the many lower valued weights. Using non uniform quantization can improve this and thereby reduce the quantization noise to improve model accuracy.

We can separate between three categories of non-uniform quantization. The simplest one is rule-based, which means the quantization levels are static and descibed by rules. The optimization based approach formulates non-uniform quantization as an optimization problem, letting the quantization levels be free variables that can be adjusted and optimized. They can for example be jointly trained with model parameters.

An example of a rule-based non-uniform quantization is logarithmic quantization. The base 2 logarithmic quantizer rule is given by (4). This equation gives a logarithmic distribution of quantization levels.

$$Q(x) = \text{sign}(x) \cdot \alpha \cdot 2^{\,q}, \qquad q = \text{clip}\Big(\text{round}\big(\log_2 \tfrac{|x|}{\alpha}\big),\, n_{\min},\, n_{\max}\Big) \tag{4}$$

Here α is the scale factor used to set the quantization range, like with uniform quantization α. Note that α does not change the quantization levels, so optimiziong α does not mean optimizing quantization levels. q is the exponent which can be stored in lower bit formats. When q is stored in fixed point it is clipped/clamped to the range that is determined by the bit-width b, typically $[-2^b - 1, 2^b]$. When using fake quantization during QAT fake clipping/clamping must also be done. Q(0) is undefined (log2(0)), and the formula neither clips nor includes 0 in the codebook.

The origional motivation I argued for non-uniform quantization was that the quantization level distribition can better be matched to the distribution of whatever you are quantizing, so we can gain accuracy. However, QAT with non-uniform quantization such as PoT might still hurt training, and reduce the model accuracy. The reason : for some levels the difference between shadow weights and quantized weights can become larger (but also some smaller)

2.6 Power of Two Quantization

Choosing base 2 logarithmic quantization lets multiplication in the MAC operations in the neural net be replaced by bit shifts. If x is an integer then $x \cdot 2^n = x << n$.

In Power of Two Quantization (PoT) the convention is to factor out from the scale as shown in (5). This is the same as normalising weights by the largest representable magnitude $N = 2^{b-1} - 1$. (Sign-magnitude, b bits: 1 sign bit s, b-1 bits for the magnitude index. One code word is reserved for zero)

$$\alpha\{2^0, 2^1, \ldots, 2^{N-1}\} = \big(\alpha\, 2^{N-1}\big)\{2^{-(N-1)}, \ldots, 2^{-1}, 2^0\}. \tag{5}$$

The result is a quantizer using negative exponents as shown in (6). Weights without the scale α gets the range $[-1, 1]$,

$$Q(x) = \alpha\ \mathrm{sign}(x)\, 2^{-k}, \qquad Q(0) = 0 \tag{6}$$

$$k = \mathrm{clip}\Big(\ \mathrm{round}\ \big(\log_2 \tfrac{\alpha}{|x|}\big),\ 0,\ 2^{b-1} - 2\Big) \tag{7}$$

$$k \in \{0, 1, \ldots, 2^{b-1} - 2\} \tag{8}$$

$$Q(x) \in \alpha \cdot \{\ 0,\ \pm 2^{-(2^{b-1}-2)},\ \ldots,\ \pm 2^{-1},\ \pm 1\ \} \tag{9}$$

2.7 (

Addetive Power of Two Quantization)

Addetive Power of Two Quantization (APoT) is an extention of PoT, that sums several powers-of-two terms to get a quantized value, $q = \alpha \cdot (2^{-k_1} + 2^{-k_2} + \cdots + 2^{-k_n})$. This gives a non-uniform, bell-shaped distribution with even finer quantization level spacing near zero. This this matches the typical bell-shaped weight distributions better than pure PoT. Similarly to PoT, APoT completely avoids multiplication in the networks MAC operations, but introduces some more computations due to summation of powers-of-two terms in the quantizer.

Primary source: [1].

3 Method

4 Results

5 Discussion

6 Conclusion

References

[1] Yuhang Li, Xin Dong, and Wei Wang. Additive Powers-of-Two Quantization: An Efficient Non-uniform Discretization for Neural Networks.

Appendices